
Reinforcement Learning under Observability: The Snake Case

Salvatore Ferracane

University of Padua, Italy
(ID: 2154255)

Abstract

The main objective of this project is to deepen the practical understanding of reinforcement learning through the design and analysis of agents based on state-of-the-art architectures. The Snake game is used as a controlled benchmark to study how architectural and modeling choices vary with the level of observability. Three agents are developed and analyzed: a Double Deep Q-Network (DDQN) for a Fully Observable Environment (FOE), a Temporal Dueling DDQN for a Partially Observable Environment (POE), and a recurrent Proximal Policy Optimization (PPO) agent to address memory limitations under partial observability. The focus is on the rationale behind these design choices and their effect on learning performance, rather than on algorithmic definitions.

1. Introduction

All agents aim to maximize cumulative reward through fruit collection and episode completion while minimizing self-collisions and boundary impacts. Rewards are shaped as $+1.0$ for winning, $+0.5$ per fruit, 0.0 per step, with penalties of -0.2 for self-collisions and -0.1 for wall impacts, promoting spatial awareness and controlled exploration.

The analysis follows a progressive approach: a deterministic heuristic establishes a performance lower bound; in the Fully Observable Environment (FOE), DDQN handles large state spaces and overestimation bias; in the Partially Observable Environment (POE) with a 5×5 local view, Temporal Dueling DDQN reduces self-collisions by separating state-value and advantage estimation over time.

A quantitative comparison evaluates the impact of observability and architecture. Finally, as Bonus, a PPO-based policy-gradient agent is developed and compared systematically with the value-based method.

2. Heuristic Baseline

The heuristic baseline agent uses a hierarchical decision-making framework combining *Optimal Pathfinding* via BFS

on a 7×7 grid and a *Safety Fallback* using Flood Fill to maximize fruit collection while minimizing terminal collisions. This balances short-term optimality with long-term survivability.

Simulation Results The heuristic baseline was evaluated over $N = 100$ boards for a maximum of $T = 1000$ steps. The performance metrics are reported in Table 1.

Metric	Value
Average Total Reward	47.1250
Standard Deviation	11.5662

Table 1. Heuristic Baseline Performance Metrics.

An average total reward of 47.1250 corresponds to ~ 94 fruits per episode, accounting for penalties (-0.1 for walls, -0.2 for self-collisions). The standard deviation of 11.5662 reflects variability from random fruit placement and occasional unavoidable collisions. These results establish a strong benchmark for subsequent learning agents.

3. Fully Observable Environment

3.1. Double DQN Design

As the cardinality of the state space $\mathcal{S}$ grows exponentially with both the board size and the snake length, tabular methods quickly become infeasible. For this reason, a deep neural network is employed to approximate the action-value function. However, standard DQN are known to suffer from systematic overestimation of action values, a phenomenon induced by the maximization operator used in the target computation (as shown in Figure 2).

Double DQN (DDQN) explicitly addresses this issue by decoupling action selection from action evaluation. In particular, the online network with parameters θ is used to select the greedy action in the next state, while the target network with parameters θ^- is used to evaluate the corresponding action value (Sutton & Barto, 2018)¹. The resulting target

¹According to the notation in Chapter 6.7 (Maximization Bias and Double Learning), θ corresponds to Q_1 (online, learnable network), while θ^- corresponds to Q_2 (target network) used for

is defined as

$$Y_t^{DDQN} = r + \gamma Q(s', \arg \max_a Q(s', a; \theta); \theta^-) \quad (1)$$

Beyond value overestimation, another critical challenge in deep RL is the violation of the Independent and Identically Distributed (IID) assumption required by stochastic gradient-based optimization. To mitigate this issue, an experience replay mechanism is adopted. In this work, a *vectorized replay buffer* is implemented to natively support the `n_boards` parallel simulation structure of the environment.

The buffer leverages NumPy-based slicing for batch insertion and sampling, enabling efficient memory access patterns and high CPU utilization². Compared to standard iterative replay buffers, this design significantly reduces training time while preserving the decorrelation properties required for stable learning.

Network Architecture To effectively exploit the full observability of the environment, the agent adopts a CNN to extract spatial features from the $7 \times 7 \times 4$ state representation. This architectural choice allows the value function to capture local geometric patterns that are directly relevant for navigation and collision avoidance.

The network begins with two 2D convolutional layers equipped with ReLU activations, which are particularly effective at identifying spatial structures such as proximity to walls, body obstructions, and free corridors. These features are then propagated through fully connected layers to estimate action values. Compared to fully connected architectures, the convolutional design significantly improves sample efficiency by exploiting translational invariances inherent to the grid-based environment.

From an optimization standpoint, several design choices are introduced to improve robustness during training. First, the Huber loss is employed instead of a standard mean squared error. This loss behaves quadratically for small temporal-difference errors and linearly for large ones, reducing sensitivity to outliers and stabilizing learning during early training stages, where reward signals are particularly noisy. Second, global gradient norm clipping with threshold 10.0 is applied to prevent exploding gradients, which can occur after high-reward transitions such as episode termination due to a win condition.

Overall, these architectural and optimization choices are aimed at ensuring stable learning dynamics while maintaining sufficient representational capacity to model the complex spatial dependencies of the Snake environment.

external evaluation.

²CPU available: AVX2 FMA

3.2. Simulation Results

Experimental Setup Experience collection uses a vectorized environment with $N = 1000$ parallel board instances, reducing temporal correlations and maximizing replay buffer throughput. Training is conducted for $T = 10,000$ iterations to allow the agent to fully transition from exploration to exploitation and to assess asymptotic stability. Random seeds are fixed to 0 for deterministic network initialization and fruit placement.

Hyperparameters Table 2 summarizes the training hyperparameters.

Parameter	Value and Rationale
Discount factor γ	0.99 (long-term survival)
Learning rate α	1×10^{-4} (stable updates)
Batch size B	256 (reduces variance)
Exploration ϵ -decay	Linear over 5,000 steps
Replay buffer capacity $ \mathcal{D} $	200,000 transitions
Parallel environments N	1000 boards
Training horizon T	10,000 iterations

Table 2. Double DQN Training Hyperparameters.

DDQN Training Procedure The training logic of the Double DQN agent follows the standard off-policy value-based paradigm, with explicit separation between action selection and action evaluation. Algorithm 1 summarizes the core update step used during training.

Algorithm 1 Double DQN Training Step

Input: minibatch of transitions (s, a, r, s', d) , discount factor γ
 Compute $Q(s, \cdot; \theta)$ using the online network
 Select greedy next action:

$$a^* \leftarrow \arg \max_{a'} Q(s', a'; \theta)$$

Evaluate next-state action value using target network:

$$Q_{\text{next}} \leftarrow Q(s', a^*; \theta^-)$$

Compute DDQN target:

$$Y \leftarrow r + (1 - d)\gamma Q_{\text{next}}$$

Compute Huber loss between Y and $Q(s, a; \theta)$

Update θ via gradient descent with clipped gradients

Periodically update target network parameters $\theta^- \leftarrow \theta$

Optimization Strategy The chosen hyperparameters balance exploration efficiency, optimization stability, and long-term performance. The high discount factor and extended

training horizon allow the agent to internalize long-term spatial constraints beyond heuristic capabilities. Large batch sizes and extensive replay memory stabilize learning by reducing gradient noise and sample correlation.

This setup provides a robust and reproducible foundation for the quantitative comparisons in the following sections, where the learned policy is evaluated against the heuristic baseline and extended to partially observable settings.

Results Training progress was evaluated using the *Average Reward per Step* ($\bar{r}_t$), smoothed over a moving average window of $w = 100$ iterations, as shown in Figure 1. The Double DQN agent consistently outperforms the heuristic baseline, achieving an improvement of approximately +160% in projected score (see Table 3). This confirms the agent’s ability to learn long-term strategies beyond the deterministic heuristic while maintaining stable and reproducible performance.

To illustrate the learning progress and performance improvement of the Double DQN agent, a demonstration is available in the following links: [Google-drive] and [Git-repository].

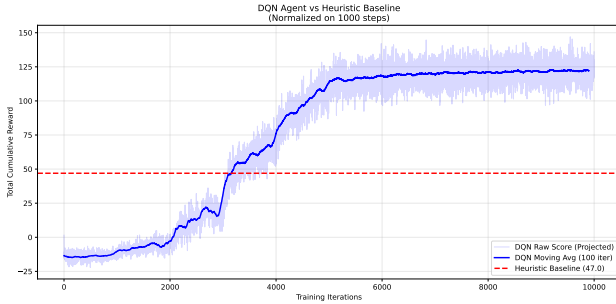


Figure 1. Training curves of Standard DQN and Double DQN compared to the Heuristic baseline, showing both raw reward and moving average.

Agent	Early R/Step	Late R/Step ($\mu \pm \sigma$)	Proj.
Heuristic	—	0.0471	47.1
Std. DQN	-0.0026	0.1213 ± 0.0074	121.3
Double DQN	-0.0045	0.1214 ± 0.0075	121.4

Table 3. Early performance (first 2000 iterations) shows faster apparent learning in Std. DQN due to overestimation bias, while Double DQN is more conservative. Late performance (8000–10000 iterations) shows identical convergence and stability. “Proj.” is the projected score over 1000 steps and, “R/Step” is Reward per step.

Early training dynamics (Fig. 1) show that Std. DQN improves faster, driven by overestimation bias that optimistically inflates early value estimates and accelerates exploitation. Double DQN removes this bias by decoupling action selection and evaluation, resulting in slower but more accurate initial learning. Despite this difference, both converge to the same performance ceiling (≈ 121 projected reward),

demonstrating that in a deterministic, fully observable MDP, overestimation accelerates learning without affecting final policy optimality.

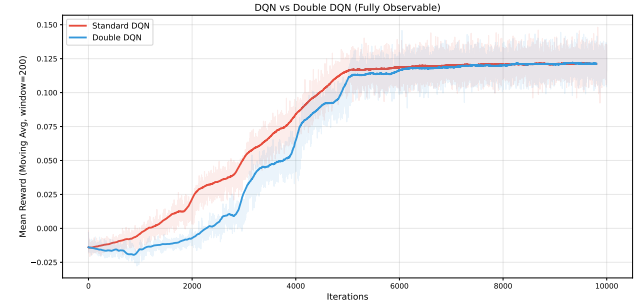


Figure 2. Comparison between Double and Standard DQN.

Convergence and Robustness The learning curve exhibits stable asymptotic convergence after approximately 5000 iterations, confirming the effectiveness of the Double DQN architecture and the target network update mechanism in stabilizing value estimation. Despite the intrinsic sparsity of the reward signal, the agent successfully learned to associate delayed rewards with preceding state–action sequences, avoiding short-sighted behaviors.

From Table 4, the robustness evaluation across 100 independent episodes and multiple random seeds demonstrates strong generalization capability, with a mean reward of 132.13 ± 3.62 and a narrow confidence interval. The near-zero wall collision rate confirms reliable spatial awareness, while the fruit collection rate (0.286 fruits/step) indicates efficient path planning consistent with shortest-path behavior under grid constraints. Residual failures are primarily due to state-space saturation in late episodes rather than instability in the learned policy.

Metric	Value
Total Episodes	100
Mean Total Reward	132.13 ± 3.62
95% Confidence Interval	[131.42, 132.84]
Min / Max Reward	123.10 / 141.50
Fruit Collection Rate	0.286 fruits/step
Wall Collision Rate	≈ 0.000 events/episode
Self-Collision Rate	0.053 events/episode

Table 4. Performance Metrics of Double DQN across 100 Evaluation Episodes by using the final weights.

4. Partially Observable Environment

4.1. Dueling Double DQN Design

Under partial observability, the environment is naturally modeled as a **Partially Observable Markov Decision Process (POMDP)**. The agent perceives only a local 5×5

observation window centered on its head, which replaces the global 7×7 state available in the fully observable setting. This restriction violates the Markov property, since the observation o_t no longer uniquely determines the underlying state s_t . In particular, global boundaries, full body topology, and future collision risks may lie outside the visible horizon. As a consequence, optimal decision-making requires inferring latent spatial structure from incomplete observations.

Spatio-Temporal State Representation To mitigate information loss, a temporal aggregation mechanism based on **frame stacking** is introduced. Instead of relying on a single observation, the agent processes a short-term history of the last $k = 4$ frames,

$$\Phi(s_t) = [o_t, o_{t-1}, o_{t-2}, o_{t-3}] \quad (2)$$

This transforms the input tensor from $(5, 5, 4)$ to $(5, 5, 16)$, allowing the convolutional encoder to extract motion-dependent features such as direction, velocity, and body evolution. This implicit memory partially restores state identifiability by embedding short-term dynamics into the representation, enabling the agent to anticipate future collisions even when critical regions are temporarily outside the visible area.

Dueling Double DQN Architecture Partial observability introduces states whose long-term outcome is largely independent of the immediate action, such as configurations leading to unavoidable collisions. To improve value estimation under these conditions, the network adopts a **Dueling Double DQN** (DDDQN) architecture (Wang et al., 2016). The action-value function is decomposed into a state-value component $V(s)$ and an action-advantage component $A(s, a)$, combined as

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right) \quad (3)$$

This decomposition allows the agent to learn which states are intrinsically safe or dangerous independently of the selected action, improving convergence speed and stability when action effects are difficult to distinguish from local observations alone.

4.2. Simulation Results

Optimization Strategy To handle the increased uncertainty of the partially observable environment, several training adjustments were introduced, as summarized in Tables 5 and 6. The agent uses an extended exploration decay of

12,000 steps to thoroughly explore ambiguous situations, while a learning rate of 1×10^{-4} and a batch size of 256 balance stable updates with computational efficiency. The Huber loss reduces sensitivity to high-variance targets, and a replay buffer of 200,000 transitions combined with 5,000 warmup steps ensures sufficient experience coverage before training. Updates occur every 4 steps, with the target network refreshed every 1,000 steps to prevent value oscillations. Frame stacking ($k = 4$) embeds short-term dynamics, and the Dueling DDQN architecture separates state-value and advantage estimation, allowing the agent to recognize risky configurations independently of actions. The total training horizon of 12,000 iterations ensures convergence across the expanded belief space.

Summary of Architectural Differences Table 5 summarizes the key differences between the two agents for fully observable (FOE) and partially observable (POE) environment.

Feature	FOE	POE
Input Dimensions	$7 \times 7 \times 4$	$5 \times 5 \times 16$ (Stacked)
Architecture	DDQN	Dueling DDQN
Decision Model	Reactive (Markovian)	Spatio-Temporal (Memory-based)
Training Horizon	10,000	12,000

Table 5. Comparison of Agent Configurations. “Stacked” indicates temporal frame stacking.

Hyperparameters Table 6 summarizes the training hyperparameters for Dueling DDQN agent.

Parameter	Value
Discount factor γ	0.99
Learning rate α	1×10^{-4}
Batch size B	256
Exploration ϵ -decay	Linear over 12,000 steps
Replay buffer capacity $ \mathcal{D} $	200,000 transitions
Warmup steps	5,000
Training frequency	Every 4 steps
Target update frequency	Every 1,000 steps
Observation processing	Frame stacking $k=4$
Network architecture	Dueling Double DQN
Training horizon T	12,000 iterations

Table 6. Dueling Double DQN hyperparameters for the Partially Observable Environment.

Results Evaluation over five random seeds (Table 7) shows consistent but severely constrained performance, with mean reward $\mu = 2.98$ and mean episode length of 24.6 steps and hence ≈ 0.121 reward per steps. The low variance across seeds confirms that the observed limitations are struc-

tural rather than stochastic. The agent reliably avoids walls, demonstrating correct learning of static local constraints. However, all episodes terminate via self-collision (100%), indicating a systematic failure in long-term survival.

Seed	Avg Re-ward	Avg Steps	Wall Coll.	Self Coll.
42	3.00	24.6	0%	100%
100	3.07	25.2	0%	100%
2024	2.40	20.5	0%	100%
777	3.40	27.9	0%	100%
99	3.02	25.0	0%	100%
Mean	2.98	24.6	0%	100%

Table 7. Evaluation metrics of the Partially Observable agent (Dueling DDQN) across five random seeds.

“Ghost Tail” Effect The primary failure mode originates from the limited 5×5 observation horizon, which introduces hidden state variables once the snake length exceeds the visible region. This violates the Markov assumption and causes perceptual aliasing, where identical local observations correspond to different global configurations. Frame stacking ($k = 4$) enables recovery of short-term motion but not the global body topology, leading the agent to select locally safe yet globally unsafe actions and resulting in inevitable self-collisions (“Ghost Tail”).

This structural limitation is reflected in the stable but low reward regime shown in Figure 3, where the mean reward quickly plateaus, and in the low variance across seeds in Figure 4, confirming that the failure is systemic rather than stochastic. A representative episode illustrating this behavior is available in [Google Drive link].

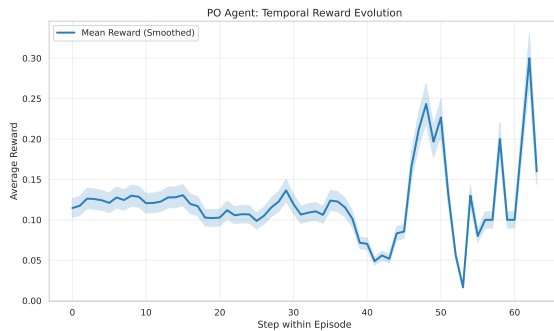


Figure 3. Mean reward per step in the partially observable environment. The early plateau indicates convergence to a suboptimal policy constrained by partial observability.

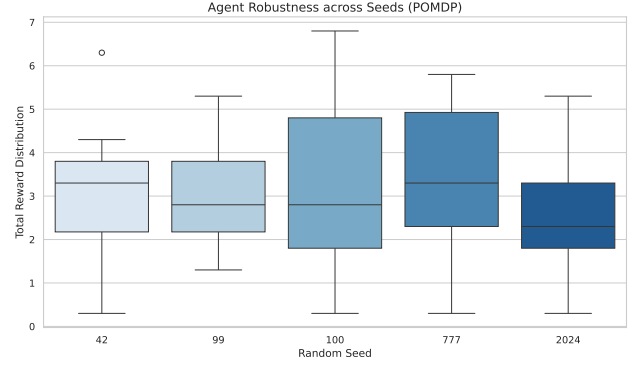


Figure 4. Reward distribution across random seeds in the partially observable environment. While internal variance remains significant due to the inherent uncertainty of the POMDP, the consistent performance ceiling across all seeds confirms that agent efficiency is structurally limited by the observation constraint rather than stochastic instability.

5. Full vs. Partial Observability

The transition from global to local observation produces a drastic performance degradation (Table 8). The mean reward decreases from 132.13 to 2.98 (−97.74%), and the mean episode length drops by over 91%. Importantly, wall avoidance remains perfect in both settings, confirming that local perception is sufficient for reactive safety but insufficient for strategic survival.

Metric	FO	PO	Gap
Mean Total Reward	132.13	2.98	−97.74%
Mean Steps/Episode	≈ 280	24.6	−91.21%
Wall Collision Rate	0%	0%	0%
Self-Collision Rate	< 5%	100%	+95.00%

Table 8. Performance comparison between Fully Observable (FO) and Partially Observable (PO) agents. Gap denotes relative change with respect to FO.

Architectural Differences Table 9 reports the main differences between Double DQN and Dueling DDQN agents.

Metric	FOE	POE
Input space	7×7 global	5×5 local
State property	Markovian	Non-Markovian
Policy type	Global planning	Local reactive
Avg reward	≈ 132	≈ 3.0
Primary failure	Exploration limits	Hidden-state collision

Table 9. Architectural comparison: Fully Observable (FO) and Partially Observable (PO) environments.

Policy Degradation The FO agent learns globally optimal behaviors that preserve future maneuverability, whereas the PO agent converges to a greedy policy that maximizes immediate reward without accounting for future spatial constraints. This behavior is optimal under local information but suboptimal in environments where survival depends on global topology.

Implications These results reveal an architectural limitation for the Dueling DQN: frame stacking offers finite memory adequate for first-order dynamics but insufficient to capture long-term spatial dependencies. To address this, a PPO agent was developed in the Bonus section, aiming to achieve improved performance under POE.

6. PPO Agent (Bonus)

To address the limitations of value-based methods under *partial observability*, a PPO agent augmented with an LSTM memory module was trained over 2,000 training iterations, 512 step per iterations and 8 parallel environments. Table 10 summarizes all training hyperparameters.

Parameter	Value
Discount factor γ	0.99
GAE parameter λ	0.95
Clip ratio	0.2
Learning rate α	3×10^{-4}
Epochs per update	4
Batch size B	64
LSTM units	128
Dense layer units	256

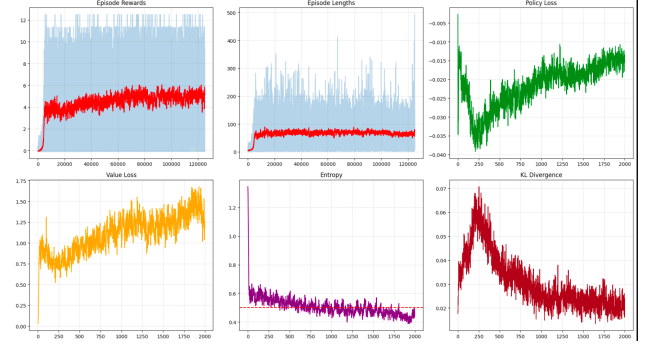
Table 10. PPO training and environment hyperparameters.

Simulation Results Table 11 summarizes the PPO performance, while Figure 5 shows stable training dynamics. Under partial observability, PPO substantially outperforms the Dueling DDQN, reducing the gap caused by missing global state information. By exploiting temporal memory, the agent maintains an internal state representation, mitigating perceptual aliasing and achieving a mean reward of 4.16. This confirms the superiority of memory-based policy optimization over reactive value-based methods in POMDPs. The final policy effectiveness is confirmed in the [gameplay GIF](#), where the snake reaches the maximum achievable length.

Metric	DDQN	PPO	DDQN
Mean Reward	2.98	4.16	132.13
Mean Steps/Episode	25	237	280

Table 11. Direct performance comparison between Dueling DDQN and the PPO agent under Partial Observability (POE) and Double DQN under FOE. The integration of memory in PPO mitigating the “ghost tail” collisions inherent to reactive architectures.

Figure 5. PPO training analysis under POE.



PPO vs. Dueling DDQN Under partial observability, the Dueling DDQN exhibits a structural limitation: relying solely on instantaneous local observations, it cannot track its full body once it exceeds the visible mask, resulting in the “ghost tail” effect and low performance ($\mu \approx 2.98$). In contrast, PPO leverages temporal memory to reconstruct hidden state information beyond the observation window, achieving higher performance ($\mu = 4.16$), a mean survival of 237.5 steps, and a peak reward of 11.30, close to the grid capacity limit. The [gameplay GIF](#), confirms that the agent performs sustained long-horizon foraging, reaching near-maximum length while maintaining global spatial consistency and effective collision avoidance despite severe partial observability.

7. Conclusion

The project analyze the impact of observability on value-based reinforcement learning by implementing Double DQN in a FOE and Dueling DDQN in a POE, enabling a direct evaluation of how state visibility influences agent design and performance.

In the FOE, the agent learned a stable near-optimal policy, achieving a mean reward of 132.13, confirming the effectiveness of DDQN under Markovian conditions. In contrast, the Dueling DDQN agent suffered a severe performance degradation, with a mean reward of 2.98 and a 100% self-collision rate, caused by hidden state information and the resulting “Ghost Tail” effect.

The benefit of recurrent memory is confirmed by the PPO agent, whose ability to reconstruct hidden state information mitigates the Ghost Tail limitation. In particular, PPO achieves a mean survival of 237 steps, approaching the FO DDQN benchmark (280 steps) and vastly exceeding the Dueling DDQN baseline (25 steps). This demonstrates that memory-based policies can recover near-optimal survival even under partial observability.

In conclusion, these results experimentally demonstrate the structural limitations of feedforward value-based methods under partial observability.

References

- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT press, 2nd edition, 2018.
- Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., and de Freitas, N. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pp. 1995–2003. PMLR, 2016.